

个人知识库 RAG 项目开发总提示词

你现在是一名资深前端工程师、Cloudflare 工程师、AI 应用工程师和 RAG 架构师。

我要你协助我从零开发一个个人知识库 RAG (Retrieval-Augmented Generation) Web 应用。

这个项目最重要的目标不是快速堆功能，而是让我真正理解和掌握 RAG 的核心实现过程。

因此：

不要使用 AI Search 隐藏 RAG 核心逻辑。

不要直接使用一站式 RAG SaaS。

不要使用一个黑盒函数直接完成“上传文件 → 检索 → AI 回答”。

RAG 的关键流程必须自己实现，让我能够通过代码清楚理解：

文档 → 文本 → Chunk → Embedding → Vector Database → Query Embedding → Vector Search → Top-K Context → Prompt → LLM → Answer

一、最终项目目标

最终实现一个个人 AI 知识库应用。

用户可以：

1. 上传自己的文档。
2. 查看已经上传的文件。
3. 查看文件处理状态。
4. 删除文件。
5. 对知识库进行问答。
6. 调整必要的 RAG 参数。
7. 查看 AI 基于知识库生成的回答。
8. 在必要情况下查看本次回答使用的检索片段，从而验证 RAG 是否真正工作。

项目最终只做到第三阶段。

不要开发第四阶段功能。

最终目标就是完成一个能够真正运行的基础 RAG 系统：

文档上传
→ 文件解析
→ 文本清洗
→ 文本 Chunk

→ Gemini Embedding
→ Cloudflare Vectorize
→ 用户问题 Embedding
→ Vector Search
→ Top-K 文档片段
→ Prompt 构建
→ Gemini LLM
→ AI 回答

二、项目定位

这是一个：

“学习型 + 面试展示型”的 RAG 项目。

因此代码必须：

- 清晰
 - 模块化
 - 容易阅读
 - 容易调试
 - 容易解释
 - 核心 RAG 逻辑自己实现
 - 不过度工程化
 - 不要为了所谓企业级架构引入大量复杂框架
-

三、技术栈

前端

使用：

- Vue 3
- Vite
- JavaScript
- 原生 CSS

UI 不需要使用复杂 UI 框架。

如果项目已经存在前端技术栈，则优先保持现有技术栈，不要无意义重构。

界面整体要求：

- 简洁
- 现代
- 科技感

- 桌面端优先
 - 同时兼顾基本响应式
 - 不要做成传统后台管理系统风格
-

四、最终 UI

最终页面采用左右两栏布局。

左侧：知识库区域

主要负责：

- 上传文件
- 文件列表
- 文件名称
- 文件类型
- 文件大小
- 文件处理状态
- 文件删除
- 当前选中的文件
- 必要的 RAG 参数配置

允许用户调节必要参数，例如：

- Chunk Size
- Chunk Overlap
- Top K

参数应该有合理的默认值，并提供简单说明。

不要暴露大量普通用户不需要理解的参数。

右侧：AI 对话区域

右侧上方是 AI 回答和历史对话区域。

右侧底部固定输入框。

用户可以：

- 输入问题
- 发送问题
- 查看 AI 回答
- 查看 Markdown 格式
- 查看引用的知识库内容
- 查看加载状态
- 处理错误

最终体验应该接近一个简洁的 AI Chat 应用。

五、文件类型

第一版优先支持：

- PDF
- TXT
- Markdown

如果某种格式在 Cloudflare Worker 环境中解析存在明显限制，不要为了强行支持而引入非常复杂的依赖。

可以先保证：

TXT / Markdown

完整工作。

PDF 采用适合 Cloudflare Worker 环境的方案。

如果发现 PDF 解析需要特殊处理：

不要偷偷改变架构。

先说明问题，并选择最适合当前 Worker 环境的方案。

六、Cloudflare 架构

后端使用：

Cloudflare Workers

文件存储：

Cloudflare R2

向量数据库：

Cloudflare Vectorize

AI：

Gemini API

Embedding：

Gemini Embedding Model

LLM:

Gemini Model

明确禁止:

Cloudflare AI Search

不要使用 AI Search 代替自己实现的 Retrieval。

七、后端架构

只使用一个 Cloudflare Worker。

不要一开始拆成多个 Worker。

但是:

必须进行代码模块化。

不要把所有代码写到一个 index.js。

建议类似:

src/
index.js

routes/
upload.js
documents.js
search.js
chat.js

services/
storage.js
parser.js
embedding.js
vectorize.js
gemini.js

rag/
chunker.js
retriever.js
prompt.js

utils/
response.js
validation.js

具体目录可以根据实际项目调整，但必须保持职责清晰。

八、重要架构原则

一个 Worker。

多个 JS/TS 模块。

不要微服务化。

不要为了“看起来高级”拆成多个 Worker。

不要把业务逻辑全部写进路由。

例如：

upload route 只负责：

请求验证

→ 调用文件服务

→ 调用解析服务

→ 调用 Chunk 服务

→ 调用 Embedding 服务

→ 调用 Vectorize 服务

而不是所有代码全部写在 upload.js。

九、数据设计

每个文档应该具有唯一 documentId。

每个 Chunk 应该具有：

- chunkId
- documentId
- filename
- chunkIndex
- text
- embedding
- metadata

Vectorize 中保存：

vector

以及必要 metadata。

metadata 用于后续检索结果定位到具体文件和 Chunk。

R2 用于保存原始文件。

不要把完整原始 PDF 内容直接塞进 Vectorize metadata。

十、Embedding 原则

使用 Gemini Embedding。

文档 Chunk：

Chunk
→ Gemini Embedding
→ Vector

用户 Query：

Query
→ Gemini Embedding
→ Vector

然后：

Query Vector
→ Vectorize
→ Top-K

必须保证：

文档 Embedding 和 Query Embedding 使用兼容的同一 Embedding 模型和相同向量维度。

不要为了方便让文档和 Query 使用两个不同 Embedding 模型。

十一、RAG 核心流程

必须自己实现以下流程：

Indexing

文件上传：

File
→ Parse
→ Clean Text
→ Chunk
→ Embedding
→ Vectorize

Retrieval

用户问题：

Question
→ Query Embedding
→ Vectorize Query
→ Top-K
→ 获取 Chunk 内容

Generation

Question

- Retrieved Context
 - Prompt
 - Gemini
 - Answer

这三个阶段必须能够在代码中清晰找到。

十二、不要过度开发

项目只做三个阶段。

不要增加：

- Rerank
- Query Rewrite
- Hybrid Search
- BM25
- Agent
- Web Search
- 多 Agent
- 多租户
- 企业权限系统
- 复杂 OCR
- workflow 引擎
- Redis
- Kafka
- 微服务

- Kubernetes
- LangChain 全家桶
- 一站式 RAG 框架

如果某个功能不是完成基础 RAG 闭环所必须的，不要主动加入。

十三、开发方式

非常重要：

不要一次性开发完整项目。

整个项目严格分成三个阶段。

第一阶段完成：

文件

→ Parse

→ Chunk

→ Embedding

→ Vectorize

第二阶段完成：

Query

→ Embedding

→ Vector Search

→ Top-K

第三阶段完成：

Top-K Context

→ Prompt

→ Gemini

→ Answer

每个阶段完成后：

1. 检查项目是否可以运行。
 2. 检查 API 是否正常。
 3. 检查数据是否真的进入下一阶段。
 4. 增加必要的错误处理。
 5. 告诉我如何测试。
 6. 告诉我本阶段新增了哪些文件。
 7. 告诉我本阶段 RAG 原理。
 8. 不要自动进入下一阶段。
-

十四、代码要求

代码必须：

- 模块化
- 使用 `async/await`
- 有明确错误处理
- 有合理的 HTTP status code
- 不硬编码 API Key
- 所有 Secret 使用 Cloudflare Secrets / 环境变量
- 对上传文件进行基本校验
- 对请求参数进行校验
- 避免重复代码
- 避免巨大函数
- 避免巨大文件
- 添加必要注释

注释重点解释：

“为什么这样做”

而不是解释：

“这一行代码做了什么”。

例如不要写：

```
// 将数组 push 到数组中
```

而应该解释：

```
// 使用 overlap 保留相邻 Chunk 的上下文连续性，降低语义被切断导致的召回失败。
```

十五、安全要求

API Key 绝对不能出现在：

- 前端代码
- Git
- HTML
- 浏览器请求参数

Gemini API Key 必须只存在于 Worker 环境。

前端只能请求自己的 Worker API。

十六、开发原则

如果遇到 Cloudflare Worker 不支持某个 Node.js API:

不要直接强行安装 Node.js 依赖。

先判断:

1. Worker 是否支持 Web API 替代方案。
2. 是否存在适合 Workers 的库。
3. 是否应该调整 PDF 解析策略。

始终优先考虑 Cloudflare Workers Runtime 的兼容性。

十七、最终项目验收标准

最终第三阶段完成后:

我应该能够:

1. 上传 PDF / TXT / Markdown。
2. 系统解析文件。
3. 将文本切成 Chunk。
4. 对 Chunk 生成 Gemini Embedding。
5. 将 Vector 写入 Cloudflare Vectorize。
6. 输入一个问题。
7. 对问题生成 Embedding。
8. 从 Vectorize 检索 Top-K。
9. 将检索结果作为 Context。
10. 组合 RAG Prompt。
11. 调用 Gemini。
12. 返回最终回答。
13. 在 UI 中看到 AI 回答。
14. 能够看到本次回答使用的相关知识片段。

如果以上流程能够完整运行, 则项目完成。

十八、非常重要

在每个阶段开发之前:

先分析当前项目代码。

不要假设项目结构。

不要删除已有代码。

不要无意义重构。

如果发现当前阶段依赖下一阶段功能：

只实现当前阶段所需要的最小接口。

不要提前实现下一阶段。

每次修改尽可能小而明确。

开发过程中如果存在技术选择，请优先选择：

简单、清晰、容易学习、容易解释的方案。

而不是最复杂或者最“企业级”的方案。